

At scale with Cluster Toolkit (gcluster)

Contents

- Overview of the workflow
- 1. Prerequisites
- 2. Environment configuration
- 3. Step-by-Step MaxText TPU Smoke Test (Multi-Host)
- 4. Run a production MaxText job on GCS datasets
- 5. Running with Ahead-of-Time (AOT) Compilation
- 6. Monitoring and managing workloads

This guide provides the recommended workflow for running MaxText on Google Kubernetes Engine (GKE) using **Cluster Toolkit's gcluster CLI**. For a complete reference on Cluster Toolkit and `gcluster`, please see the [official Cluster Toolkit repository](#) and [Google Cloud documentation](#).

Overview of the workflow

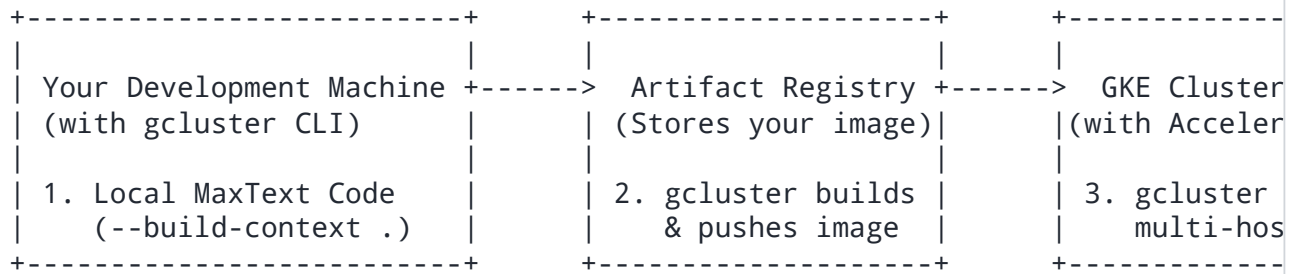
The process involves two main stages:

1. **Packaging or Layering the MaxText Application:** You can either prebuild a MaxText Docker image or use `gcluster`'s built-in Crane integration (`--build-context .`) to automatically build and layer your local MaxText working directory on top of a base image on-the-fly.
2. **Orchestrating Workload Deployment:** `gcluster job submit` automatically discovers your GKE cluster's accelerator topology, verifies Kueue and JobSet health, generates the required Kubernetes resources (`JobSet`, `ClusterQueue`, `LocalQueue`, `ResourceFlavor`), and deploys your multi-host JAX workload.



Последние





1. Prerequisites

Before you begin, ensure you have the necessary client tools installed and permissions configured.

Required tools

- **Python** `>= 3.12` with `pip` and `venv`.
- **Google Cloud CLI** (`gcloud`): Install it from [here](#) and run `gcloud init`.
- **kubectl & GKE auth plugin:**

```
gcloud components install kubectl gke-gcloud-auth-plugin
```

- **gcluster CLI:** Follow the official [Cluster Toolkit CLI Installation Guide](#) (or download the binary from the [releases page](#)) and ensure `gcluster` is in your `$PATH`.
- **Docker Credentials:** Configure Docker authentication for your target Artifact Registry region:

```
gcloud auth configure-docker <region>-docker.pkg.dev --quiet
```

GCP permissions and APIs

Make sure the required Google Cloud APIs are enabled on your project:

```
gcloud services enable \
  container.googleapis.com \
  artifactregistry.googleapis.com \
  storage.googleapis.com
```

Your Google Cloud user account needs the following IAM roles in your target project:

- Artifact Registry Writer
- Kubernetes Engine Admin / Developer
- Storage Admin (for accessing GCS buckets for checkpoints and datasets)
- Logging / Monitoring Viewer

2. Environment configuration

First, ensure your local `kubect1` is authenticated with your target GKE cluster:

```
# Google Cloud Configuration
export PROJECT_ID=<PROJECT_ID>
export LOCATION=<ZONE> # e.g., 'europe-west4' or 'us-east5-a'
export GKE_CLUSTER=<CLUSTER_NAME>

# Workload Configuration
# Note: Workload names cannot exceed 28 characters and must be valid DNS labels
export RUN_NAME=<RUN_NAME>

# Hardware & Slice Configuration
export COMPUTE_TYPE=ct5p-hightpu-4t # e.g., 'ct5p-hightpu-4t' for v5p, or 'c2p-hightpu-4t' for v4p
export TOPOLOGY=4x4x4 # e.g., '4x4x4' (64 chips)

# MaxText & Storage Configuration
export BASE_OUTPUT_DIRECTORY=<GCS_BUCKET> # e.g., gs://my-bucket/output
export DATASET_PATH=<DATASET_PATH> # e.g., gs://my-bucket/dataset
export DOCKER_IMAGE="us-docker.pkg.dev/cloud-tpu-images/maxtext-images/tpu_p..."
```

Next, configure your default Google Cloud project, target GKE cluster name, and cluster location in `gcluster`:

```
gcloud config set project ${PROJECT_ID?}
gcloud container clusters get-credentials ${GKE_CLUSTER?} \
  --location ${LOCATION?} \
  --project ${PROJECT_ID?}
gcluster job config set project ${PROJECT_ID?}
gcluster job config set cluster ${GKE_CLUSTER?}
gcluster job config set location ${LOCATION?}
```

When `gcluster` runs, it inspects your GKE cluster and verifies that **Kueue** and **JobSet** custom resource definitions and webhooks are healthy. If Kueue  **Последние** 
`gcluster` can prompt to install it automatically.

3. Step-by-Step MaxText TPU Smoke Test (Multi-Host)

To verify that your GKE cluster, Kueue admission queues, and multi-host JAX ICI interconnects are fully functional before launching large production runs, you can run a lightweight synthetic smoke test using MaxText's `default` dummy model (`model_name=default`).

```
cd /path/to/maxtext

gcluster submit \
  --base-image=
  --build-context=. \
  --command=
  --name=
  --compute-type=${COMPUTE_TYPE?} \
  --topology=${TOPOLOGY?}
```

4. Run a production MaxText job on GCS datasets

This section assumes you have an existing GKE cluster with TPU or GPU node pools.

Шаг 1. Экспорт репозитория целевых изображений

Укажите, в каком репозитории Artifact Registry в целевом проекте и регионе должен храниться образ контейнера, созданный с помощью `gcluster`. Если у вас еще нет репозитория Docker в Artifact Registry, создайте его:

```
gcloud artifacts repositories create maxtext-images \
  --repository-format=docker \
  --location=<GCP_REGION> \
  --description="Образы контейнеров MaxText"
```

Более подробную информацию можно найти в официальной [документации по реестру артефактов](#). После создания экспортируйте имя реп



Последние



```
= GCLUSTER_IMAGE_REPOexportmaxtext-images
```

Шаг 2. Отправьте обучающую рабочую нагрузку

Перейдите в корневой каталог вашего локального репозитория MaxText и отправьте задание с помощью `gcluster job submit`.

При использовании `--build-context .` пакетов ваш локальный исходный каталог MaxText размещается поверх облегченного базового образа (`--base-image`) и образ контейнера создается/обновляется на лету. Это позволяет быстро тестировать локальные изменения кода без необходимости вручную создавать образ контейнера Docker.

(Примечание: если у вас уже есть полностью собранный пользовательский образ контейнера, содержащий ваш код, загруженный в Artifact Registry, вы можете указать

`--image=${IMAGE_NAME?}` напрямую вместо `--base-image` и `--build-context`.)

```
cd /path/to/maxtext
gcluster job submit \
  --base-image=${DOCKER_IMAGE?} \
  --build-context=. \
  --command="python3 -m maxtext.trainers.pre_train.train run_name=${RUN_NAME?}" \
  --name=${RUN_NAME?} \
  --compute-type=${COMPUTE_TYPE?} \
  --topology=${TOPOLOGY?}
```

Ключевые флаги:

- `--compute-type`: тип ускорителя (например, `ct5p-hightpu-4t` для TPU v5p или `ct6e-hightpu-4t` для TPU v6e).
- `--topology`: физическая топология 3D-сетки, необходимая для сегментов TPU с несколькими хостами (например, `4x4x4` распределяет 64 чипа между 16 физическими узлами; `2x2x1` распределяет 4 чипа между 1 узлом).
- `--build-context`: путь к локальной директории сборки (`.`), позволяющий быстро итеративно тестировать локальные изменения в MaxText.

5. Запуск с предварительной компиляцией (AOT)

Ahead-of-Time (AOT) компиляция может значительно сократить время запуска вашего обучающего задания за счет предварительной компиляции этапа обучения JAX для конкретной топологии аппаратного обеспечения. С помощью `gcluster` вы можете сгенерировать скомпилированный pickle-файл локально и подключить его через `--build-context .`

Шаг 1. Создание артефакта AOT

Обратите внимание, что для локального запуска `train_compile.py` требуется среда Python с установленными MaxText и JAX. Прежде чем запускать компиляцию, настройте локальную среду в соответствии с [руководством по началу работы с MaxText](#).

Запустите `train_compile.py` локально, чтобы создать скомпилированный артефакт для целевой топологии TPU:

```
1 =PER_DEVICE_BATCH_SIZE <TPU_TYPE> export
1 =NUM_SLICESexport
# например, "v5p-128" =TPU_TYPEexport

python3 -m maxtext.trainers.pre_train.train_compile \
  compile_topology=${TPU_TYPE?} \
  compile_topology_num_slices=${NUM_SLICES?} \
  compiled_trainstep_file=maxtext_${TPU_TYPE?}_aot.pickle \
  per_device_batch_size=${PER_DEVICE_BATCH_SIZE?}
```

Это создаст `maxtext_${TPU_TYPE?}_aot.pickle` в корневом каталоге вашего репозитория MaxText.

Шаг 2. Отправьте рабочую нагрузку, ссылаясь на артефакт AOT

При запуске `gcluster job submit` с `--build-context .` сгенерированный файл pickle автоматически упаковывается в образ контейнера. Передайте

`compiled_trainstep_file=maxtext_${TPU_TYPE?}_aot.pick`

 Последние ▼

загрузил предварительно скомпилированный шаг:

```
gcluster job submit \  
  --base-image=${DOCKER_IMAGE?} \  
  --build-context=. \  
  --command="python3 -m maxtext.trainers.pre_train.train run_name=${RUN_NAME?}" \  
  --name=${RUN_NAME?} \  
  --compute-type=${COMPUTE_TYPE?} \  
  --topology=${TOPOLOGY?}
```

6. Мониторинг и управление рабочими нагрузками

Список заданий в кластере

Просмотр активных рабочих нагрузок или рабочих нагрузок в очереди в кластере:

```
gcluster job list
```

Проверить статус пода

Список всех подов, относящихся к отправленной вами рабочей нагрузке:

```
kubectl get pods -l gcluster.google.com/workload=${RUN_NAME?}
```

Просмотр журналов рабочей нагрузки

Потоковая передача журналов из ведущего контейнера с помощью `gcluster` или `kubectl`:

```
gcluster job logs ${RUN_NAME?}  
# Или с помощью kubectl:  
kubectl logs -f -l gcluster.google.com/workload=${RUN_NAME?},jobset.sigs.k8s
```

Или воспользуйтесь ссылкой на Google Cloud Console Logs Explorer, которую можно найти с помощью `gcluster job submit`.

  [Последние](#) ▼

Отменить или удалить задание

Чтобы завершить выполняющуюся рабочую нагрузку или очистить завершённые ресурсы набора заданий:

```
gcluster job cancel ${RUN_NAME?}
```

Или удалите Kubernetes JobSet напрямую:

```
kubectl delete jobset ${RUN_NAME?}
```